

第21讲 练习题与答案

Context Window: 上下文窗口与长期记忆

使用建议

先独立完成前两页。做题时请一直区分“应用层保存了什么”和“模型这一次 forward 实际看到了什么”。

1. 定义

用自己的话解释 Context Window。

2. Token Budget

总窗口 64K，如果系统+历史+工具+检索材料用了 52K，还想预留 8K 输出，大致还剩多少 token 给当前新增输入？

3. Context vs Memory

为什么外部数据库里保存了10年历史，不代表模型这一轮自动知道全部10年历史？

4. KV Cache

KV Cache 保存的是什么？为什么它能加速 Decode？

5. Long Context Cost

为什么输入从8K变到128K时，通常不能认为成本和延迟完全不变？

6. Capacity vs Usability

“支持1M上下文”和“能可靠利用1M中任意位置证据”为什么是两个问题？

7. Lost in the Middle

设计一个最简单的位置敏感性实验。

8. RAG

为什么一个有1M窗口的模型仍可能需要 RAG？

9. Agent Context Bloat

列出至少3种 Agent 运行时容易制造大量无用 token 的来源。

10. Multi-Agent

为什么 Full Broadcast 在 Agent 数增加后可能迅速变贵？

11. Memory Design

给一个长期科研 Agent 设计“工作上下文 + 外部记忆 + 检索”的最小结构。

12. 科研指标

如果研究 context management，你会报告哪些指标？

答案与解释

1.

一次推理时模型当前能够同时接收和利用的 token 工作区上限。它是临时计算上下文，不等于长期存储。

2.

约 4K。64K - 52K - 8K = 4K。实际还要考虑模板、特殊 token 和具体 API 限制。

3.

数据库只是外部存储。模型要在当前推理中使用其中信息，必须由 runtime 检索、选择并重新放入当前 context。

4.

通常保存各层历史 token 的 Key/Value 表示。Decode 新 token 时可复用过去 K/V，避免每一步把整个前缀重新计算一遍。

5.

更长输入增加 prefill 工作、注意力交互、KV cache、显存和很多服务的 token 计费；具体增长速度依架构和实现而不同。

6.

前者是容量声明，后者涉及检索、位置鲁棒性、抗干扰、多步推理和系统效率，需要单独测试。

7.

保持问题和证据不变，只把关键证据分别放在开头、1/4、中间、3/4、结尾；其余干扰材料尽量一致，重复多次并比较准确率。

8.

RAG 可以先筛掉无关信息，降低 token 成本、TTFT 和干扰，并从超大知识库中按需选择资料。窗口很大并不意味着应该把所有资料都塞进去。

9.

完整工具日志、长网页原文、每轮 reflection、重复 system/tool schema、所有历史消息、Agent 间全量广播等。

10.

每个 Agent 产生的消息都可能被复制给其余 Agent；通信量和上下文重复会随 Agent 数和轮数一起增长，进一步放大 token 与延迟。

11.

例如：当前任务状态和最近几轮放 working context；历史实验/结论写入结构化 store 或向量库；每轮按 query 检索 Top-k 相关条目，压缩后注入 context。

12.

至少包括任务成功率、context length、输入/输出

token、成本、TTFT/总延迟、检索召回率、关键信息保留率、位置敏感性、摘要错误率/冗余率等。

加餐：判断题

- A. 1M Context 意味着模型会永久记住这一轮所有内容。
- B. 历史消息被应用保存，不代表每次调用都一定完整重放。
- C. KV Cache 和向量数据库是同一类 Memory。
- D. Long Context 和 RAG 可以同时使用。
- E. 多 Agent 系统通信越多，context bloat 风险通常越低。

参考：A错，B对，C错，D对，E错。